

完整场景链路 验证材料

八条端到端链路，从一句诉求走到业务终态，每一步都落在同一套事件契约与状态机上。

全部证据可由第三方在本机一条命令重放核验。

这份材料要回答的只有一个问题：**凭什么相信这些链路真的跑过？**

答案不是截图，是 **163 条状态迁移**、**314 条事件**、**418 个 span** 的原始记录，加上一个把它们逐项重放的核验器 —— 它当前的输出是 **RESULT: 8/8 PASS**。

8

端到端链路

8/8

核验项 PASS

67

Skill 真实调用

1935

存量测试通过

代码基线 bcd5d467f63b310dafeff4de38a351e9ca0a7d70 (goai-restructure)

证据基线 cfe438475f751bb1a7bfe034daf7b2381314888d

核验结论 RESULT: 8/8 PASS | exit=0

测试结论 1935 passed, 41 skipped in 52.17s

生成时间 2026-09-03T16:11:54+08:00

复现命令 python3 scripts/make_evidence.py && python3 scripts/verify.py

证据目录 evidence/scenario-1..7 · evidence/scenario-R5 · evidence/room

先说清楚这份材料证明什么、不证明什么 —— 边界不划清，后面每一个数字都可以被读成别的意思。

✓ 这份材料证明

- ① 八条链路各自真的从头跑到了终态，中间每一次状态变化都在库里有行；
- ② 每个 Skill / ToolPort 调用都留了 input_digest 与 output_hash，与事件日志逐条对得上；
- ③ 判「业务成功」的依据是外部权威判据（沙箱 pytest 报告 / 支付网关回执），不是 Agent 自己说完成了；
- ④ 这些结论可以被外人独立重放 —— 核验器八项逐项 PASS，任一项失败都说得出失败意味着什么。

× 这份材料不证明

- ① 不证明业务数据是真实客户数据 —— 政策与历史案例是按行业惯例构造的合成数据，支付网关的错误码与时序取自公开规范；
- ② 不证明模型效果 —— 缺省路径用 ScriptedModelClient，token 数是估算，只能读作调用规模；
- ③ 不证明退款真的到账 —— 权威在支付网关，MAOS 只持有观察与推断；
- ④ 不证明性能 —— 八条链路合计 5947 ms 是确定性复现的耗时，不是压测结论。

§0.2 八条链路各自在证明什么

场景	链路	这一条单独拿出来证明的那件事
1	正常闭环	基准线成立：一条无异常的链路在这套契约上能从头走到尾，且每步留痕
2	返工闭环	失败会被闸挡住，并且带着结构化 findings 回到执行侧 —— 返工是有输入的
3	高风险人工审批	「闸过了」不等于「可以放行」，风险档位能独立把任务钉在人工停点上
4	幂等验证	重复投递不会造出第二次状态迁移 —— 幂等是数得出来的，不是声称的
5	治理路径（重规划）闭环	计划本身可以被推翻重排，且重排走 Plan 级迁移、不抹掉前一轮记录
6	制造企业售后退款 · 顺利路径	同一套内核换到退款域零改动，外部权威判据从 pytest 换成网关回执
7	制造企业售后退款 · 失败路径	业务没成功时系统如实记录：不猜终态、不写 settled、留补偿与人工工单
R5	RAG 有无对照实验	检索不是装饰 —— 关掉它，同一份脚本会漏排任务并被闸判死

§0.3 怎么读这份材料

§1	验证方法	两条命令 · 核验器八项判据分别在防什么	3
§2	核验实跑输出	本次运行的原始输出全文，未经编辑	4
§3	八条链路总表	一张表看完 span / 事件 / 调用 / 终态	5
§4	公共骨架	8 个状态 · 16 条合法迁移 · 七道闸	6
§5.1– 5.8	逐条链路验证	任务 DAG → 调用序列 → 迁移轨迹 → 判据 → 终态	7–16
§6	Skill 调用面	13 个 Skill × 版本 × 真实调用次数 × 九要素	17
§7	ToolPort 与哈希完整性	4 个 ToolPort · 93 组摘要逐条比对	18
§8	权威事实边界	场景 6 与 7 的对照：成功与失败凭什么分开	19
§9	RAG 有无对照	同一份计划脚本，只切一个开关	20
§10	人机协同实证	真 Matrix 房间四种结局	21
§11	复现清单	第三方从零复现的完整步骤与文件清册	22

一条贯穿全文的判准

本文所有计数都从 evidence/ 下的 SQLite 库与 JSON 直接读出，由生成脚本写入，全文没有手打的数字。证据文件首行一律是 « generated at <ISO8601> from <git sha> »。

检索不准顶多说效果一般；无法核验就是零分。所以每一项判据都必须能被外人独立跑一遍，且失败时说得
出失败意味着什么。

§1.1 两条命令，缺一不可

```
$ python3 scripts/make_evidence.py # ① 跑 7 个场景 + RAG 对照，产 evidence/scenario-1..7 与 -R5
$ python3 scripts/verify.py        # ② 八项证据逐项重放校验 → RESULT: 8/8 PASS
```

为什么不能只跑第 ② 条：*.db 不入 git（被 .gitignore 挡着），核验器要的是库、不是快照。直接跑 ② 会报「缺数据库」并退出 2 —— 这是设计行为，不是坏了。全新克隆实测 clone + 两条命令共约 7 秒。

§1.2 八项判据分别在防什么

#	判据	校验什么	这一项失败意味着
1	hash-integrity	每个 skill / tool 调用的 input_digest、output_hash 与 event_log 一致	证据被篡改，或事后手写
2	business-ref	每条 business_ref 指向的对象在库中存在且 version 匹配	引用悬空 —— 业务锚点是假的
3	authoritative-fact	每个 settled 都有对应 payment_observation，且 actor_invocation_id 属于 payment.observe	权威事实边界被绕过
4	trace-tree	trace.json 的 span 树无孤儿、无环，且与库重放逐字节一致	事件链不完整
5	kb-hit	每个 KbRetrieved 的 doc_id 在 kb_doc 中存在	RAG 命中是编的
6	business-outcome	每个 Plan 终态都有 business_outcome；DONE 必须有外部判据，且那条判据能在库里回查得到	「Agent 都完成了」被当成业务成功
7	history-case	本库晋升的 history_case 都能追溯到 outcome=success 的真实 case	知识层被污染
8	cost-attribution	每条 model_usage 都挂得到 plan 的 trace_id、task_id 回查得到，estimated 标记与 model 列相符	成本归因是假的，或估算被印成了真实计费

§1.3 三种结论与退出码

结论	进不进分子	什么时候出现	退出码
[PASS]	进	该项有可判对象，且全部通过	0（全 PASS 时）
[FAIL]	进（记为不通过）	有对象且判不过 —— 逐条点名，不给汇总糊过去	非 0
[SKIP]	不进	上游能力没落地，或该项分母为 0（空转）	不单独影响

不许静默跳过

一个 7/7 里藏着两个没跑的，比老老实实写 5/5 PASS + 2 SKIP 更坏 —— 后者看得出没跑。

空转也算没跑

0/0 PASS 与「真跑了且全过」在屏幕上长得一模一样。这是核验器能犯的最坏的错，所以专门做了 _idle_skip 挡它。

依赖方向

核验器不 import maos/domain/** 的业务逻辑，只按「表在不在」决定某一项跑还是 SKIP —— 换一个业务域，这八项一行都不用改。

下面是本次运行的原始输出全文，未经编辑。带 **info / warn** 的行是核验器主动说出口的保留意见——它们不影响 PASS，但故意留在屏幕上。

```
$ python3 scripts/verify.py
[PASS] hash-integrity      93/93
[PASS] business-ref       35/35
[PASS] authoritative-fact 3/3
    · info: scenario-7 case=case-s7-0001: 有回执且案子收口在 biz_status=compensated (非 settled 终态) —— 预期行为: settled 是权威终态, 收口在别处的案子本来就不该有 settled 观察
    · warn: scenario-7 case=case-s7-0002: 有回执但案子停在中间态 biz_status=gateway_accepted —— 观察到了但没收口 (既没到 settled, 也没落到 ['compensated', 'rejected'] 任何一个终态)
[PASS] trace-tree         29/29
    · info: scenario-1: 2 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.reviewer.review_after_gate; maos.flows.common.patch_verifier。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
    · info: scenario-2: 3 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.reviewer.review_after_gate; maos.flows.common.patch_verifier。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
    · info: scenario-3: 1 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.testing.seed_scripted_report。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
    · info: scenario-5: 2 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.testing.seed_scripted_report。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
    · info: scenario-6: 1 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.reviewer.review_after_gate。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
    · info: scenario-7: 1 份产物走旁路入库 (未经 on_task_result), 来源已由 ArtifactSeeded 事件点名: maos.agents.reviewer.review_after_gate。这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode
[PASS] kb-hit             7/7
[PASS] business-outcome   10/10
[PASS] history-case       1/1
[PASS] cost-attribution   57/57

RESULT: 8/8 PASS
证据来源: scenario-1, scenario-2, scenario-3, scenario-4, scenario-5, scenario-6, scenario-7, scenario-R5
verify exit=0
```

读这两行保留意见

scenario-7 case-s7-0001 有网关回执但收口在 compensated —— 判 info: settled 是权威终态, 收口在别处的案子本来就不该有 settled 观察。
case-s7-0002 停在 gateway_accepted —— 判 warn: 观察到了但没收口。这是如实记录, 不是缺陷被藏起来。

「旁路入库」那六行

有产物未经 on_task_result 入库, 来源已由 ArtifactSeeded 事件逐个点名。核验器把话说死: 这说的是入库路径, 不是内容真伪 —— 判真伪看 sandbox.mode。

§2.2 同一基线上的存量测试

```
$ python3 -m pytest maos/tests -q
..... [100%]
1935 passed, 41 skipped in 52.17s
```

证据核验与单元测试是两条独立的绳子: 前者验「跑过的这一次留下的记录站不站得住」, 后者验「代码在各种输入下的行为对不对」。两条都绿才算数, 任一条红都不许提交。

八条链路一张表。每一格都从 `evidence/<场景>/maos.db` 与 `trace.json` 直接读出，没有一个数字是手打的。

场景	链路	域	Plan	任务	迁移	Span	事件	Skill	Tool	产物	耗时	Plan 终态
1	正常闭环	软件交付域	1	4	16	37	26	4	2	6	1447ms	DONE
2	返工闭环	软件交付域	1	4	21	48	35	6	3	8	1937ms	DONE
3	高风险人工审批	软件交付域	1	1	5	17	12	2	1	3	423ms	DONE
4	幂等验证	软件交付域	1	1	3	10	7	2	1	1	408ms	RUNNING
5	治理路径（重规划）闭环	软件交付域	1	1	9	32	26	8	2	4	560ms	DONE
6	制造企业售后退款·顺利路径	退款域	1	5	21	54	41	9	3	7	360ms	DONE
7	制造企业售后退款·失败路径	退款域	2	8	41	103	80	17	8	13	392ms	FAILED FAILED
R5	RAG 有无对照实验	退款域·对照	3	14	47	117	87	19	6	13	420ms	DONE FAILED DONE
合计	八条链路·两个业务域·同一套内核		11	38	163	418	314	67	26	55	5947ms	7 DONE / 3 FAILED

§3.2 3 个 FAILED 不是缺陷，是这份材料的重点

失败的 Plan 各自在证明什么

场景 7 两个 Plan 都 FAILED —— 网关问不出终态，主管驳回；

R5 · without_kb FAILED —— 关掉检索后计划漏排财务核算，第六道闸判 blocker；

一条只演成功路径的系统，等于没演。

7 个 DONE 的判据不是「都完成了」

每个 DONE 的 `business_outcome.basis` 都是 `external_evidence`，且那条判据在库里回查得到：软件域是沙箱 `pytest` 报告，退款域是 `payment_observation` 里的 `observed_state=settled`。

§3.3 八条链路合计产出的事件（按类型）

事件类型	条数	它证明链路上发生了什么
StateTransition	163	任务状态变了一次 —— 带 <code>from_state</code> / <code>to_state</code> / <code>reason</code> ，是链路的骨架
SkillInvoked	67	一次 Skill 调用，带版本、 <code>invocation_id</code> 与输入输出摘要
ToolInvoked	26	一次 ToolPort 调用，带 <code>params_digest</code> —— 摸外部世界的唯一出口
PlanTransition	25	Plan 级状态变了一次（含 <code>replan</code> 回到 PENDING）
RefundBizStatusChanged	17	业务对象自己的状态变了 —— 不是 Task 状态（铁律 9）
ArtifactSeeded	10	产物走旁路入库，来源被事件点名 —— 不许有无名产物
Replanned	2	计划被推翻重排了一次
KbRetrieved	2	一次知识检索，候选文档与打分全留
CompensationAttached	1	任务挂上了补偿路径（第五道闸的判据）
CompensationExecuted	1	域内补偿真的执行了

合计 314 条，10 种类型。事件类型集合冻结在 `maos/contracts/events.py`，受三重机制强制（deny 规则 · PreToolUse 守卫 hook · 指纹校验测试）。

八条链路走的是同一副骨架：8 个任务状态、16 条合法迁移、七道闸。迁移表在 `contracts/states.py` 里冻结 —— 表里没有的迁移，控制面直接抛 `IllegalTransition`。

§4.1 任务状态机（8 态 / 16 迁移）

从	到	reason	走到
PEND	DISP	dispatch	38
DISP	RUN	claim	38
DISP	PEND	claim_timeout	—
RUN	AWAIT	submit_result	38
RUN	BLOCK	worker_blocked	—
RUN	PEND	retry	—
RUN	FAIL	retry_exhausted	—
AWAIT	DONE	gate_pass	25
从	到	reason	走到
AWAIT	REWK	gate_rework	3
AWAIT	BLOCK	gate_needs_human	9
AWAIT	FAIL	gate_reject_final	—
REWK	PEND	requeue	3
REWK	FAIL	rework_exhausted	—
BLOCK	PEND	human_resume	—
BLOCK	DONE	human_approve	6
BLOCK	FAIL	human_reject	3

合计 163 次状态迁移，覆盖 9 / 16 条合法迁移。没走到的那几条（`claim_timeout`、`retry`、`rework_exhausted` 等）是超时与耗尽路径，确定性复现的链路里本来就不该出现 —— 它们由单元测试覆盖，不靠场景演。

§4.2 七道闸（按顺序跑，任何一道不过就出 rework）

#	闸	判据	作用域
1	schema	产物结构合规	所有任务
2	acceptance	代码类看沙箱测试报告，非代码类看 <code>self_check</code> ；没有报告 = blocker，无降级	所有任务
3	security	敏感面变更的安全判据	所有任务
4	evidence	结论必须挂得上证据	所有任务
5	compensation	有副作用就必须有补偿路径	所有任务
6	finance	退款金额超阈值 → 必须有财务核算凭据；plan 级判据还管「漏排财务复核」	退款域
7	gateway	认网关回执，按官方码表判四象限处置，产出 <code>replan</code> 换渠道的输入	退款域

闸是规则驱动，不是模型驱动

七道闸没有一道调模型。判据写死在 `maos/runtime/gate.py` 里，同一份输入永远得到同一个结论 —— 一个会飘的闸挡不住任何东西。

闸的结论是三态，不是两态

`pass` / `noted` / `fail`。`noted` 是「这道闸有话说但没拦」—— 压成 `pass` 就把它藏起来了。

幂等键的口径（场景 7 的关键）

退款的幂等键由（`tenant`, `case`）定 —— 换渠道不换键。所以 40005 触发换渠道重发时不会造出第二笔；而 `outcome=unknown` 那一格对重规划一票否决，因为「能不能再发一次」和「这一笔到底执行了没有」是两个正交的维度。

需求 → 架构 → 编码 → 沙箱回归，四个任务一次过闸，Plan 收在 DONE。

37
span

26
事件

16
状态迁移

4
Skill 调用

2
Tool 调用

6
产物

1447ms
端到端耗时

Plan 修复 demo/app 的 token 校验缺失 DONE plan_fb2978440926 · trace_135a1ed9cf76

角色	任务	终态	attempt	risk
requirement	归一化目标并给出验收标准	DONE	1	L
architecture	产出架构契约	DONE	1	L
coding	修复 demo/app 的 token 校验缺失	DONE	1	L
testing	沙箱回归验证	DONE	1	M

返工 0 · 重规划 0 · 补偿 0 · Skill 4 · Tool 2 · 事件 26 · Plan 耗时 1064ms

业务终态 succeeded · basis = external_evidence

沙箱测试报告 art_8a09155a9789 (passed=5, 来源 artifact_seeded)

沙箱测试报告 art_a997ade7e503 (passed=5, 来源 task_result)

调用 调用序列（按 event_log 的 seq 排，共 6 步）

01 S req.normalize29ms

02 S kb.retrieve0ms

03 T git-mcp134ms

04 S code.repo-patch135ms

05 T sandbox.pytest_run334ms

06 S test.verify335ms

S = SkillInvoked (带版本 · invocation_id · input_digest · output_hash) T = ToolInvoked (带 params_digest)

产物 产物（6 份，每份都挂在具体任务上）

task_id	kind	version	artifact_id
s1-req	requirement	1	art_a7bdbc7581da
s1-arch	architecture_contract	1	art_7f1869269272
s1-code	patch_set	1	art_2377e13eb2ac
s1-code	test_report	1	art_8a09155a9789
s1-test	test_report	1	art_a997ade7e503
s1-test	review_note	1	art_aa1dc0af249c

事件 本条链路的事件类型分布（26 条 / 5 种）

StateTransition 16

SkillInvoked 4

PlanTransition 2

ToolInvoked 2

ArtifactSeeded 2

轨迹 状态迁移（库内原始记录，共 16 条）

s1-req	PEND	→ DISP	dispatch	s1-req	DISP	→ RUN	claim
s1-req	RUN	→ AWAIT	submit_result	s1-req	AWAIT	→ DONE	gate_pass
s1-arch	PEND	→ DISP	dispatch	s1-arch	DISP	→ RUN	claim
s1-arch	RUN	→ AWAIT	submit_result	s1-arch	AWAIT	→ DONE	gate_pass
s1-code	PEND	→ DISP	dispatch	s1-code	DISP	→ RUN	claim
s1-code	RUN	→ AWAIT	submit_result	s1-code	AWAIT	→ DONE	gate_pass
s1-test	PEND	→ DISP	dispatch	s1-test	DISP	→ RUN	claim
s1-test	RUN	→ AWAIT	submit_result	s1-test	AWAIT	→ DONE	gate_pass

看点

这是基准线：没有返工、没有停点、没有补偿的那条路长什么样。后面七条都是在它身上加异常。

外部权威判据：沙箱 pytest 测试报告 | 证据目录 evidence/scenario-1/ (result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db)

第 1 轮用例真的挂了，Gate 判 rework 并回结构化 findings，第 2 轮修好过闸。

48
span

35
事件

21
状态迁移

6
Skill 调用

3
Tool 调用

8
产物

1937ms
端到端耗时

Plan 修复 demo/app 的会话过期判定 DONE plan_bf8fe96ca5d4 · trace_a4c8f08478ba

角色	任务	终态	attempt	risk
requirement	归一化目标并给出验收标准	DONE	1	L
architecture	产出架构契约（第一轮不完整）	DONE	1	L
coding	修复 demo/app 的会话过期判定	DONE	2	L
testing	沙箱回归验证	DONE	1	M

返工 1 · 重规划 0 · 补偿 0 · Skill 6 · Tool 3 · 事件 35 · Plan 耗时 1587ms

业务终态 succeeded · basis = external_evidence

沙箱测试报告 art_75202122c865 (passed=5, 来源 task_result)

沙箱测试报告 art_4b6a901886c1 (passed=5, 来源 artifact_seeded)

调用 调用序列（按 event_log 的 seq 排，共 9 步）

01 S req.normalize 17ms 02 S kb.retrieve 0ms 03 T git-mcp 135ms

04 S code.repo-patch 137ms 05 S kb.retrieve 0ms 06 T git-mcp 132ms

07 S code.repo-patch 134ms 08 T sandbox.pytest_run 323ms 09 S test.verify 324ms

S = SkillInvoked (带版本 · invocation_id · input_digest · output_hash) T = ToolInvoked (带 params_digest)

产物 产物 —— 注意 s2-code 的 version 1 与 2 并存（返工留痕）

task_id	kind	version	artifact_id
s2-req	requirement	1	art_f056aba14f2c
s2-arch	architecture_contract	1	art_c7382215dc32
s2-code	patch_set	1	art_0a6ee6395f90
s2-code	test_report	1	art_3b803bd17760
s2-code	patch_set	2	art_4ce4d2e7483f
s2-code	test_report	2	art_4b6a901886c1
s2-test	test_report	1	art_75202122c865
s2-test	review_note	1	art_8ba482d27b94

事件 本条链路的事件类型分布（35 条 / 5 种）

StateTransition 21

SkillInvoked 6

ToolInvoked 3

ArtifactSeeded 3

PlanTransition 2

轨迹 状态迁移（库内原始记录，共 21 条）

s2-req PEND → DISP dispatch s2-req DISP → RUN claim s2-req RUN → AWAIT submit_r...

s2-req AWAIT → DONE gate_pass s2-arch PEND → DISP dispatch s2-arch DISP → RUN claim

s2-arch RUN → AWAIT submit_r... s2-arch AWAIT → DONE gate_pass s2-code PEND → DISP dispatch

s2-code DISP → RUN claim s2-code RUN → AWAIT submit_r... s2-code AWAIT → REWK gate_rew...

s2-code REWK → PEND queue s2-code PEND → DISP dispatch s2-code DISP → RUN claim

s2-code RUN → AWAIT submit_r... s2-code AWAIT → DONE gate_pass s2-test PEND → DISP dispatch

s2-test DISP → RUN claim s2-test RUN → AWAIT submit_r... s2-test AWAIT → DONE gate_pass

看点

返工不是重跑一遍，是带着 findings 重跑。编码任务 attempt 从 1 变 2，两轮产物各留一份、版本号不同 —— 第一轮那份不许删。

七道闸全过，任务照样停在 BLOCKED —— 要不要人拍板由风险档位决定，不由闸的结论决定。

17

span

12

事件

5

状态迁移

2

Skill 调用

1

Tool 调用

3

产物

423ms

端到端耗时

Plan 高风险变更需人工放行 DONE plan_f4a140c48f5e · trace_2fb8069495e8

角色	任务	终态	attempt	risk
coding	变更生产环境配置	DONE	1	M

返工 0 · 重规划 0 · 补偿 1 · Skill 2 · Tool 1 · 事件 12 · Plan 耗时 156ms

业务终态 succeeded · basis = external_evidence

沙箱测试报告 art_d7ddd3533cb8 (passed=1, 来源 artifact_seeded)

调用 调用序列 (按 event_log 的 seq 排, 共 3 步)

01 S kb.retrieve 16ms 02 T git-mcp 124ms 03 S code.repo-patch 126ms
S = SkillInvoked (带版本 · invocation_id · input_digest · output_hash) T = ToolInvoked (带 params_digest)

实录 run.log 实录 (真实终端输出)

```
待人工审批: ['变更生产环境配置']
INFO maos.cp [task_25f1efb0b583] 已附着补偿引用 -> attempt=1
INFO maos.cp [task_25f1efb0b583] RUNNING -> AWAITING_REVIEW (submit_result)
INFO maos.gate [task_25f1efb0b583] Gate {'schema': 'pass', 'acceptance': 'pass', 'security': 'pass',
'evidence': 'pass', 'compensation': 'pass', 'finance': 'pass', 'gateway': 'pass'} -> pass
INFO maos.cp [task_25f1efb0b583] AWAITING_REVIEW -> BLOCKED (gate_needs_human)
INFO maos.cp [task_25f1efb0b583] BLOCKED -> DONE (human_approve)
```

产物 产物 (3 份)

task_id	kind	version	artifact_id
task_25f1efb0b583	test_report	1	art_d7ddd3533cb8
task_25f1efb0b583	patch_set	1	art_0e661dde63de
task_25f1efb0b583	compensation	0	art_731b826e32c8

事件 本条链路的事件类型分布 (12 条 / 6 种)

StateTransition 5 PlanTransition 2 SkillInvoked 2 ArtifactSeeded 1 ToolInvoked 1 CompensationAttached 1

轨迹 状态迁移 (库内原始记录, 共 5 条)

task_25f1efb0b...	PEND	→ DISP	dispatch	task_25f1efb0b...	DISP	→ RUN	claim
task_25f1efb0b...	RUN	→ AWAIT	submit_result	task_25f1efb0b...	AWAIT	→ BLOCK	gate_needs_human
task_25f1efb0b...	BLOCK	→ DONE	human_approve				

看点

闸 pass 与放行是两件事。这条链路把它们拆开: gate_needs_human 之后必须有 human_approve 才继续。

外部权威判据: 人工审批记录 | 证据目录 evidence/scenario-3/ (result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db)

同一条 TaskResult 重复投递 2 次，第 2 次被短路，新增日志条数 = 0。

10

span

7

事件

3

状态迁移

2

Skill 调用

1

Tool 调用

1

产物

408ms

端到端耗时

Plan 幂等验证 RUNNING plan_6d53a27ad6f8 · trace_5df5da5b1884

角色	任务	终态	attempt	risk
coding	幂等测试任务	AWAITING_REVIEW	1	L
返工 0 · 重规划 0 · 补偿 0 · Skill 2 · Tool 1 · 事件 7 · Plan 耗时 5ms				

业务终态 failed · basis = plan_not_terminal

external_evidence 0 条 —— 没有外部判据就不许判成功，MAOS 在这里什么都不写。

调用 调用序列（按 event_log 的 seq 排，共 3 步）

01 S kb.retrieve 15ms 02 T git-mcp 124ms 03 S code.repo-patch 126ms
S = SkillInvoked（带版本 · invocation_id · input_digest · output_hash） T = ToolInvoked（带 params_digest）

实录 run.log 实录 —— 幂等的判据在第一行

```
=====
重复投递 2 次 TaskResult, 新增日志条数 = 0 (期望 0)
INFO maos.worker Worker w1 启动, 可插拔 Agent: ['ap_control', 'ap_intake', 'ap_match', 'ap_treasury',
'architecture', 'claim_adjudicator', 'claim_intake', 'claim_payment', 'claim_settlement', 'coding',
'investigation_cancel', 'investigation_classify', 'investigation_intake', 'investigation_observe',
'refund_channel', 'refund_finance', 'refund_intake', 'refund_payment', 'refund_policy', 'requirement',
'reviewer', 'testing']
INFO maos.cp 创建计划 plan_6d53a27ad6f8, 共 1 个任务
INFO maos.cp [task_87f58ccee86a] PENDING -> DISPATCHED (dispatch)
INFO maos.cp [task_87f58ccee86a] DISPATCHED -> RUNNING (claim)
INFO maos.cp [task_87f58ccee86a] RUNNING -> AWAITING_REVIEW (submit_result)
INFO maos.cp [task_87f58ccee86a] 重复 TaskResult, 短路
INFO maos.cp [task_87f58ccee86a] 重复 TaskResult, 短路
```

产物 产物（1 份 —— 重复投递没有造出第二份）

task_id	kind	version	artifact_id
task_87f58ccee86a	patch_set	1	art_20566c716841

事件 本条链路的事件类型分布（7 条 / 4 种）

StateTransition 3

SkillInvoked 2

PlanTransition 1

ToolInvoked 1

轨迹 状态迁移（库内原始记录，共 3 条）

task_87f58ccee... PEND → DISP dispatch task_87f58ccee... DISP → RUN claim
task_87f58ccee... RUN → AWAIT submit_result

看点

幂等的判据不是「结果一样」，是「一条状态迁移都不许多出来」。库里数得出来，所以这条判据是机器判的。

外部权威判据：事件日志计数（期望 0） | 证据目录 evidence/scenario-4/（result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db）

多源信号聚合 8 条 findings → 3 个 issue（合并重复 5 条），选定方案后重规划闭环。

32
span

26
事件

9
状态迁移

8
Skill 调用

2
Tool 调用

4
产物

560ms
端到端耗时

Plan 修复：支付回调未校验签名 DONE plan_7f0cb575c558 · trace_cb414a36b855

角色	任务	终态	attempt	risk
coding	支付回调接入（方案乙）	DONE	2	L

返工 1 · 重规划 1 · 补偿 0 · Skill 8 · Tool 2 · 事件 26 · Plan 耗时 273ms

业务终态 succeeded · basis = external_evidence

沙箱测试报告 art_b0e788ee9452 (passed=2, 来源 artifact_seeded)

沙箱测试报告 art_01f8c5e0f495 (passed=2, 来源 artifact_seeded)

调用 调用序列（按 event_log 的 seq 排，共 10 步）

01 S	issue.aggregate	0ms	02 S	kb.retrieve	0ms	03 T	git-mcp	121ms
04 S	code.repo-patch	123ms	05 S	kb.retrieve	0ms	06 T	git-mcp	126ms
07 S	code.repo-patch	128ms	08 S	kb.sink	0ms	09 S	kb.sink	0ms
10 S	kb.sink	0ms						

S = SkillInvoked（带版本 · invocation_id · input_digest · output_hash） T = ToolInvoked（带 params_digest）

实录 run.log 实录（真实终端输出）

多源信号聚合：聚合 8 条 findings -> 3 个 issue (合并重复 5 条); blocker 1 / major 2 / minor 0

产物 产物（4 份）

task_id	kind	version	artifact_id
task-s5-payment-001	test_report	1	art_b0e788ee9452
task-s5-payment-001	patch_set	1	art_9081e3a335a3
task-s5-payment-001	test_report	2	art_01f8c5e0f495
task-s5-payment-001	patch_set	2	art_f9b5802e0392

事件 本条链路的事件类型分布（26 条 / 6 种）

StateTransition 9

SkillInvoked 8

PlanTransition 4

ArtifactSeeded 2

ToolInvoked 2

Replanned 1

轨迹 状态迁移（库内原始记录，共 9 条）

task-s5-paymen...	PEND	→ DISP	dispatch	task-s5-paymen...	DISP	→ RUN	claim
task-s5-paymen...	RUN	→ AWAIT	submit_result	task-s5-paymen...	AWAIT	→ REWK	gate_rework
task-s5-paymen...	REWK	→ PEND	queue	task-s5-paymen...	PEND	→ DISP	dispatch
task-s5-paymen...	DISP	→ RUN	claim	task-s5-paymen...	RUN	→ AWAIT	submit_result
task-s5-paymen...	AWAIT	→ DONE	gate_pass				

看点

重规划走的是 Plan 级迁移 RUNNING → PENDING [replan]，不是把任务重置一遍 —— 前一轮的记录原样留着。

外部权威判据：沙箱 pytest 测试报告 | 证据目录 evidence/scenario-5/（result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db）

多源诉求聚合 → 按下单锁定的政策版本裁定 → 财务核算（第六道闸）→ 主管放行 → 付款 → 轮询问出 settled。

54

span

41

事件

21

状态迁移

9

Skill 调用

3

Tool 调用

7

产物

360ms

端到端耗时

Plan 处理客户对轴承订单的退款诉求：多源诉求已到，需按下单当时的政策核定并退款

DONE

plan_d51ccffe89c6 · trace_c689609a0446

角色	任务	终态	attempt	risk
refund_intake	受理多源退款诉求并聚合证据	DONE	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	DONE	1	L
refund_finance	核算退款金额并写财务分录	DONE	1	M
refund_payment	发起退款并观察网关终态	DONE	1	M
refund_intake	通知客户退款结果	DONE	1	L

返工 0 · 重规划 0 · 补偿 0 · Skill 9 · Tool 3 · 事件 41 · Plan 耗时 53ms

业务终态 succeeded · basis = external_evidence

网关观察 gw_2278853ea4614ecc code=10000 → settled

调用 调用序列（按 event_log 的 seq 排，共 12 步）

01 S	kb.retrieve	1ms	02 S	issue.aggregate	0ms	03 S	refund.intake	2ms
04 S	policy.match	0ms	05 S	policy.match	0ms	06 S	finance.settle	3ms
07 T	gateway.refund	0ms	08 S	payment.execute	3ms	09 T	gateway.query	0ms
10 T	gateway.query	0ms	11 S	payment.observe	1ms	12 S	notify.customer	0ms

S = SkillInvoked（带版本 · invocation_id · input_digest · output_hash） T = ToolInvoked（带 params_digest）

实录 run.log 实录（真实终端输出）

待主管审批：核算退款金额并写财务分录

语义审查：金额按订单锁定的政策 v1 核算，依据 AS-01@v1；退款尚未发起，可放行

产物 产物（7 份，按 kind × version 折叠）

refund_case_draft v1 1

refund_finance_settlement v1 1

refund_notification v1 1

refund_payment_receipt v1 1

refund_payment_request v1 1

refund_policy_decision v1 1

review_note v1 1

事件 本条链路的事件类型分布（41 条 / 7 种）

StateTransition 21

SkillInvoked 9

RefundBizStatusChanged 4

ToolInvoked 3

PlanTransition 2

KbRetrieved 1

ArtifactSeeded 1

轨迹 状态迁移（库内原始记录，共 21 条）

task-s6-in...	PEND	→ DISP	dispatch	task-s6-in...	DISP	→ RUN	claim	task-s6-in...	RUN	→ AWAIT	submit_r...
task-s6-in...	AWAIT	→ DONE	gate_pass	task-s6-po...	PEND	→ DISP	dispatch	task-s6-po...	DISP	→ RUN	claim
task-s6-po...	RUN	→ AWAIT	submit_r...	task-s6-po...	AWAIT	→ DONE	gate_pass	task-s6-fi...	PEND	→ DISP	dispatch
task-s6-fi...	DISP	→ RUN	claim	task-s6-fi...	RUN	→ AWAIT	submit_r...	task-s6-fi...	AWAIT	→ BLOCK	gate_nee...
task-s6-fi...	BLOCK	→ DONE	human_ap...	task-s6-pa...	PEND	→ DISP	dispatch				

其余 7 条见 evidence/scenario-6/run.log 与 maos.db · event_log。

看点

与场景 1—5 是同一个 Manager、同一套 contracts/ 与 core/，换域只换了 Skill / ToolPort / 业务对象。

外部权威判据：支付网关回执 gateway_code=10000 / observed_state=settled | 证据目录 evidence/scenario-6/ (result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db)

网关先回 40005（换渠道重发），备用渠道回 ACQ.SYSTEM_ERROR，轮询三次仍非终态 → 主管驳回 → 域内补偿。

103

span

80

事件

41

状态迁移

17

Skill 调用

8

Tool 调用

13

产物

392ms

端到端耗时

Plan 1/2 处理客户对轴承订单的退款诉求：政策与金额均无异议，但支付渠道回执异常 **FAILED**

plan_6b954ee47d7f · trace_a636c8729599

角色	任务	终态	attempt	risk
refund_intake	受理多源退款诉求并聚合证据	DONE	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	DONE	1	L
refund_finance	核算退款金额并写财务分录	DONE	1	M
refund_payment	发起退款并观察网关终态（改派备用渠道）	FAILED	2	M

返工 1 · 重规划 1 · 补偿 0 · Skill 10 · Tool 6 · 事件 49 · Plan 耗时 56ms

业务终态 failed · basis = plan_failed

external_evidence 0 条 —— 没有外部判据就不许判成功，MAOS 在这里什么都不写。

Plan 2/2 处理第二笔轴承退款诉求：政策与金额均无异议，但网关判定该交易不存在 **FAILED**

plan_443cbbf8c95f · trace_16ff12ff231d

角色	任务	终态	attempt	risk
refund_intake	受理第二笔退款诉求	DONE	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	DONE	1	L
refund_finance	核算第二笔退款金额并写财务分录	DONE	1	M
refund_payment	发起第二笔退款并观察网关终态	FAILED	1	M

返工 0 · 重规划 0 · 补偿 0 · Skill 7 · Tool 2 · 事件 31 · Plan 耗时 40ms

业务终态 failed · basis = plan_failed

external_evidence 0 条 —— 没有外部判据就不许判成功，MAOS 在这里什么都不写。

调用 调用序列（按 event_log 的 seq 排，共 25 步）

01 S	issue.aggregate	0ms	02 S	refund.intake	3ms	03 S	policy.match	0ms
04 S	policy.match	0ms	05 S	finance.settle	0ms	06 T	gateway.refund	0ms
07 S	payment.execute	3ms	08 T	gateway.query	0ms	09 S	payment.observe	1ms
10 T	gateway.refund	0ms	11 S	payment.execute	1ms	12 T	gateway.query	0ms
13 T	gateway.query	0ms	14 T	gateway.query	0ms	15 S	payment.observe	1ms
16 S	refund.compensate	1ms	17 S	issue.aggregate	0ms	18 S	refund.intake	2ms
19 S	policy.match	0ms	20 S	policy.match	0ms	21 S	finance.settle	0ms
22 T	gateway.refund	0ms	23 S	payment.execute	3ms	24 T	gateway.query	0ms
25 S	payment.observe	1ms						

S = SkillInvoked（带版本 · invocation_id · input_digest · output_hash） T = ToolInvoked（带 params_digest）

实录 run.log 实录（真实终端输出）

[1] 待主管审批：核算退款金额并写财务分录
语义审查：金额按订单锁定的政策 v1 核算，依据 AS-01@v1；退款尚未发起，可放行

[2] 第七道闸认出网关回执：code=40005 retrieable=True outcome=failed -> disposition=replan_channel
网关回执 40005（调用频次超限）：retrieable=True / outcome=failed — 网关在入口就拒了，这一笔业务确定没执行，可以换渠道重发。官方处置：降低请求并发量
→ 触发 replan 重规划：付款任务换渠道 s7-primary -> s7-backup；幂等键由（tenant，case）定，换渠道不换键，不会造出第二笔

[3] 备用渠道付款回执：observed_state=unknown（问了 3 次仍非终态）
网关判据：code=ACQ.SYSTEM_ERROR retrieable=True outcome=unknown
官方处置：保持参数不变重试或查询执行结果
出处：https://aipay.alipay.com/docs/mobile-app-pay/ai-app-pay/alipay-trade-refund.html#业务错误码
重规划：已换渠道 1 次（replan 上限 MAOS_MAX_REPLAN 默认 2）；这一格 outcome=unknown 对 replan 一票否决，**不再换第三个渠道** — 重发可能真退出第二笔，正确出口是人工

[4] 待主管处置：发起退款并观察网关终态（改派备用渠道） — 回执非终态，不能当成功放行

[5] 域内补偿：作废 1 笔请求，最后观察到的下落 = unobserved (**不是 failed** — 网关自己都没给出结论)
人工工单：MT-case-s7-0001 -> 沈思锴

事件 本条链路的事件类型分布（80 条 / 8 种）

StateTransition 41 SkillInvoked 17 ToolInvoked 8 PlanTransition 6 RefundBizStatusChanged 5

ArtifactSeeded 1 Replanned 1 CompensationExecuted 1

轨迹 状态迁移（库内原始记录，共 41 条）

task-s7-in...	PEND	→ DISP	dispatch	task-s7-in...	DISP	→ RUN	claim	task-s7-in...	RUN	→ AWAIT	submit_r...
task-s7-in...	AWAIT	→ DONE	gate_pass	task-s7-po...	PEND	→ DISP	dispatch	task-s7-po...	DISP	→ RUN	claim
task-s7-po...	RUN	→ AWAIT	submit_r...	task-s7-po...	AWAIT	→ DONE	gate_pass	task-s7-fi...	PEND	→ DISP	dispatch
task-s7-fi...	DISP	→ RUN	claim	task-s7-fi...	RUN	→ AWAIT	submit_r...	task-s7-fi...	AWAIT	→ BLOCK	gate_nee...
task-s7-fi...	BLOCK	→ DONE	human_ap...	task-s7-pa...	PEND	→ DISP	dispatch	task-s7-pa...	DISP	→ RUN	claim
task-s7-pa...	RUN	→ AWAIT	submit_r...	task-s7-pa...	AWAIT	→ REWK	gate_rew...	task-s7-pa...	REWK	→ PEND	requeue
task-s7-pa...	PEND	→ DISP	dispatch	task-s7-pa...	DISP	→ RUN	claim	task-s7-pa...	RUN	→ AWAIT	submit_r...
task-s7-pa...	AWAIT	→ BLOCK	gate_nee...	task-s7-pa...	BLOCK	→ FAIL	human_re...	task-s7b-i...	PEND	→ DISP	dispatch
task-s7b-i...	DISP	→ RUN	claim	task-s7b-i...	RUN	→ AWAIT	submit_r...	task-s7b-i...	AWAIT	→ DONE	gate_pass
task-s7b-p...	PEND	→ DISP	dispatch	task-s7b-p...	DISP	→ RUN	claim	task-s7b-p...	RUN	→ AWAIT	submit_r...
task-s7b-p...	AWAIT	→ DONE	gate_pass	task-s7b-f...	PEND	→ DISP	dispatch	task-s7b-f...	DISP	→ RUN	claim
task-s7b-f...	RUN	→ AWAIT	submit_r...	task-s7b-f...	AWAIT	→ BLOCK	gate_nee...	task-s7b-f...	BLOCK	→ DONE	human_ap...
task-s7b-p...	PEND	→ DISP	dispatch	task-s7b-p...	DISP	→ RUN	claim	task-s7b-p...	RUN	→ AWAIT	submit_r...
task-s7b-p...	AWAIT	→ BLOCK	gate_nee...	task-s7b-p...	BLOCK	→ FAIL	human_re...				

产物 产物（13 份，按 kind × version 折叠）

refund_case_draft v1 2 refund_finance_settlement v1 2 refund_payment_receipt v1 2

refund_payment_receipt v2 1 refund_payment_request v1 2 refund_payment_request v2 1

refund_policy_decision v1 2 review_note v1 1

看点

四个 Agent 全部回复「完成」，而这一单没有成功 — settled 观察 0 条，业务状态收在 compensated，全程没进 settled。

外部权威判据：支付网关回执 outcome=unknown（问不出终态） | 证据目录 evidence/scenario-7/（result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db）

同一份计划脚本逐字节相同，只切 MAOS_KB_ENABLED=0/1，看检索到底改变了什么。

117
span

87
事件

47
状态迁移

19
Skill 调用

6
Tool 调用

13
产物

420ms
端到端耗时

Plan 1/3 处理客户对轴承订单的退款诉求：多源诉求已到，需按下单当时的政策核定并退款 DONE

plan_de19c8751168 · trace_8394445c6d28

角色	任务	终态	attempt	risk
refund_intake	受理多源退款诉求并聚合证据	DONE	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	DONE	1	L
refund_finance	核算退款金额并写财务分录	DONE	1	M
refund_payment	发起退款并观察网关终态	DONE	1	M
refund_intake	通知客户退款结果	DONE	1	L

返工 0 · 重规划 0 · 补偿 0 · Skill 8 · Tool 3 · 事件 38 · Plan 耗时 50ms

业务终态 succeeded · basis = external_evidence
网关观察 gw_ac72ebafbd104eec code=10000 → settled

Plan 2/3 处理客户对轴承订单的退款诉求：多源诉求已到，需按下单当时的政策核定并退款 FAILED

plan_e28678ae0fd6 · trace_1a248a40e93b

角色	任务	终态	attempt	risk
refund_intake	受理多源退款诉求并聚合证据	FAILED	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	PENDING	0	L
refund_payment	发起退款并观察网关终态	PENDING	0	M
refund_intake	通知客户退款结果	PENDING	0	L

返工 0 · 重规划 0 · 补偿 0 · Skill 2 · Tool 0 · 事件 9 · Plan 耗时 20ms

业务终态 failed · basis = plan_failed
external_evidence 0 条 —— 没有外部判据就不许判成功，MAOS 在这里什么都不写。

Plan 3/3 处理客户对轴承订单的退款诉求：多源诉求已到，需按下单当时的政策核定并退款 DONE

plan_ae58baeb09f9 · trace_2bdd6a4948cd

角色	任务	终态	attempt	risk
refund_intake	受理多源退款诉求并聚合证据	DONE	1	L
refund_policy	按下单锁定的政策版本裁定退款资格	DONE	1	L
refund_finance	核算退款金额并写财务分录	DONE	1	M
refund_payment	发起退款并观察网关终态	DONE	1	M
refund_intake	通知客户退款结果	DONE	1	L

返工 0 · 重规划 0 · 补偿 0 · Skill 9 · Tool 3 · 事件 40 · Plan 耗时 55ms

业务终态 succeeded · basis = external_evidence
网关观察 gw_694236a74b544a8f code=10000 → settled

实录 run.log 实录（真实终端输出）

```
[1/3] 准备段：跑一条完整成功的退款 case，收口后按晋升规则沉淀知识
Plan DONE, 业务状态 settled, 客户 ack 1 条
晋升：kb-r5-history-0001 kind=history_case outcome=success source_case_id=case-r5-hist
[2/3] without_kb: MAOS_KB_ENABLED=0, 计划漏排财务核算
4 个任务, KbRetrieved 0 条, 第六道闸 blocker, Plan FAILED, 业务状态 submitted
第六道闸：计划缺陷 blocker（漏排财务核算，付款前拿不到金额凭据）
第三出口：受理多源退款诉求并聚合证据 -> BLOCKED (await=human_decision, 机器返工修不好，转人工)
主管裁决：驳回 -> 计划收敛到 FAILED, 付款一次都没派发
[3/3] with_kb: MAOS_KB_ENABLED=1, 同一份计划脚本
5 个任务, KbRetrieved 1 条, 第六道闸 pass, Plan DONE, 业务状态 settled
人工审批停点：['核算退款金额并写财务分录']
差异：delta_tasks=['refund_finance.refund_finance'], 触发文档=[('kb-r5-history-0001', 0.719031), ('kb-policy-
tnt-mfg-a-AS-002-v1', 0.410541), ('kb-policy-tnt-mfg-a-AS-004-v1', 0.15), ('kb-policy-tnt-mfg-a-AS-001-v1',
0.1)]
检索漏斗：库存 17 条 -> 同租户 9 条 -> 七维预过滤后 5 条 (with_kb 段实测候选集 5 条)
```

调用 调用序列（按 event_log 的 seq 排，共 25 步）

01 S	issue.aggregate	0ms	02 S	refund.intake	1ms	03 S	policy.match	1ms
04 S	policy.match	1ms	05 S	finance.settle	0ms	06 T	gateway.refund	0ms
07 S	payment.execute	4ms	08 T	gateway.query	0ms	09 T	gateway.query	0ms
10 S	payment.observe	2ms	11 S	notify.customer	0ms	12 S	issue.aggregate	0ms
13 S	refund.intake	6ms	14 S	kb.retrieve	1ms	15 S	issue.aggregate	0ms
16 S	refund.intake	1ms	17 S	policy.match	1ms	18 S	policy.match	1ms
19 S	finance.settle	0ms	20 T	gateway.refund	0ms	21 S	payment.execute	6ms
22 T	gateway.query	0ms	23 T	gateway.query	0ms	24 S	payment.observe	2ms
25 S	notify.customer	0ms						

S = SkillInvoked（带版本 · invocation_id · input_digest · output_hash） T = ToolInvoked（带 params_digest）

事件 本条链路的事件类型分布（87 条 / 6 种）

StateTransition 47

SkillInvoked 19

RefundBizStatusChanged 8

PlanTransition 6

ToolInvoked 6

KbRetrieved 1

轨迹 状态迁移（库内原始记录，共 47 条）

task-hist-...	PEND	→ DISP	dispatch	task-hist-...	DISP	→ RUN	claim	task-hist-...	RUN	→ AWAIT	submit_r...
task-hist-...	AWAIT	→ DONE	gate_pass	task-hist-...	PEND	→ DISP	dispatch	task-hist-...	DISP	→ RUN	claim
task-hist-...	RUN	→ AWAIT	submit_r...	task-hist-...	AWAIT	→ DONE	gate_pass	ask-hist-f...	PEND	→ DISP	dispatch
ask-hist-f...	DISP	→ RUN	claim	ask-hist-f...	RUN	→ AWAIT	submit_r...	ask-hist-f...	AWAIT	→ BLOCK	gate_nee...
ask-hist-f...	BLOCK	→ DONE	human_ap...	ask-hist-p...	PEND	→ DISP	dispatch	ask-hist-p...	DISP	→ RUN	claim
ask-hist-p...	RUN	→ AWAIT	submit_r...	ask-hist-p...	AWAIT	→ DONE	gate_pass	task-hist-...	PEND	→ DISP	dispatch
task-hist-...	DISP	→ RUN	claim	task-hist-...	RUN	→ AWAIT	submit_r...	task-hist-...	AWAIT	→ DONE	gate_pass
task-nokb...	PEND	→ DISP	dispatch	task-nokb...	DISP	→ RUN	claim	task-nokb...	RUN	→ AWAIT	submit_r...
task-nokb...	AWAIT	→ BLOCK	gate_nee...	task-nokb...	BLOCK	→ FAIL	human_re...	task-kb-in...	PEND	→ DISP	dispatch
task-kb-in...	DISP	→ RUN	claim	task-kb-in...	RUN	→ AWAIT	submit_r...	task-kb-in...	AWAIT	→ DONE	gate_pass
task-kb-po...	PEND	→ DISP	dispatch	task-kb-po...	DISP	→ RUN	claim	task-kb-po...	RUN	→ AWAIT	submit_r...
task-kb-po...	AWAIT	→ DONE	gate_pass	ask_6866ba...	PEND	→ DISP	dispatch	ask_6866ba...	DISP	→ RUN	claim
ask_6866ba...	RUN	→ AWAIT	submit_r...	ask_6866ba...	AWAIT	→ BLOCK	gate_nee...	ask_6866ba...	BLOCK	→ DONE	human_ap...
task-kb-pa...	PEND	→ DISP	dispatch	task-kb-pa...	DISP	→ RUN	claim	task-kb-pa...	RUN	→ AWAIT	submit_r...
task-kb-pa...	AWAIT	→ DONE	gate_pass	task-kb-no...	PEND	→ DISP	dispatch	task-kb-no...	DISP	→ RUN	claim
task-kb-no...	RUN	→ AWAIT	submit_r...	task-kb-no...	AWAIT	→ DONE	gate_pass				

看点

关 KB 那段漏排了财务核算，第六道闸判 blocker、Plan FAILED；开 KB 那段多出 refund_finance 一个任务，收在 settled。

外部权威判据：支付网关回执 + 两段 DAG 差异 | 证据目录 evidence/scenario-R5/ (result.json · trace.json · run.log · business-objects.json · kb-hits.json · maos.db · dag-diff.json)

注册表里 13 个 Skill，八条链路上真实调用 67 次。每一次都在 `event_log` 里留 `SkillInvoked`，带版本、`invocation_id`、`input_digest`、`output_hash`、`duration_ms`。

Skill	版本	调用	ok	用时中位	出现在链路	这个 Skill 干什么
<code>policy.match</code>	1.0.0	10	10	0ms	6, 7, R5	按下单当时锁定的政策版本裁定资格，回规则编号
<code>kb.retrieve</code>	1.1.0	9	9	0ms	1, 2, 3, 4, 5, 6, R5	七维预过滤 + 多通道打分的知识检索
<code>code.repo-patch</code>	1.0.0	7	7	128ms	1, 2, 3, 4, 5	在仓库上产出补丁集
<code>issue.aggregate</code>	1.0.0	7	7	0ms	5, 6, 7, R5	多源信号去重聚合成 issue
<code>refund.intake</code>	1.0.0	6	6	2ms	6, 7, R5	多源退款诉求聚合（复用 <code>issue.aggregate</code> ）
<code>payment.execute</code>	1.0.0	6	6	3ms	6, 7, R5	向支付网关发起退款
<code>payment.observe</code>	1.0.0	6	6	1ms	6, 7, R5	轮询网关问终态 —— 唯一有权写 <code>settled</code> 的调用面
<code>finance.settle</code>	1.0.0	5	5	0ms	6, 7, R5	核算退款金额并写财务分录
<code>kb.sink</code>	1.0.0	3	3	0ms	5	把收口成功的 case 按晋升规则沉淀成知识
<code>notify.customer</code>	1.0.0	3	3	0ms	6, R5	通知客户，ack 未确认标 <code>needs_followup</code> 但不阻塞
<code>req.normalize</code>	1.0.0	2	2	29ms	1, 2	把目标归一化并给出验收标准
<code>test.verify</code>	1.0.0	2	2	335ms	1, 2	沙箱回归验证
<code>refund.compensate</code>	1.0.0	1	1	1ms	7	域内补偿：作废请求、留最后观察、开人工工单

§6.2 每个 Skill 必须声明的九要素

要素	说的是什么	缺了会怎样
① <code>purpose</code>	用途 —— 一句话说清它负责哪一段	调用方按语感猜用途，同一件事出现两个实现
② <code>input_schema</code>	输入字段与类型	输入摘要没法算，哈希完整性这一项失去意义
③ <code>output_schema</code>	输出字段与类型	下游拿不准该读哪个字段，靠 <code>try/except</code> 兜
④ <code>preconditions</code>	前置条件 —— 少哪个字段就不该调	半截输入被送进去，失败在更深的地方才炸
⑤ <code>depends_tools</code>	依赖哪些 ToolPort	看不出这个 Skill 会不会摸外部世界
⑥ <code>failure_policy / max_retries</code>	失败了重试还是上报，重试几次	失败姿态不统一，自旋与静默吞掉都可能
⑦ <code>security_boundary</code>	明确不碰什么 —— 例如补偿类一律写不出 <code>settled</code>	权威事实边界靠自觉，不靠约束
⑧ <code>reuse_note</code>	换域时这个 Skill 该怎么复用	换域时照抄一份，两份实现从此各自演化
⑨ <code>owner_roles</code>	归属角色 —— 白名单外调用一律 <code>PermissionDenied</code>	谁都能调，拒绝也不留痕

67 / 67 次调用 `status=ok`

失败的调用不会被删掉 —— 它们照样进 `event_log`，只是 `status` 不是 `ok`。

跨域复用的证据

`issue.aggregate` 被调用 7 次，横跨软件交付域与退款域，同一个版本号、同一份实现；`kb.retrieve` 同理，9 次。能复用的连 Skill 都不用换。

Skill 通过 ToolPort 摸外部世界。四个 ToolPort 合计调用 26 次 —— 每一次都留 `params_digest`，与 `event_log` 逐条比对。

ToolPort	调用	用时中位	干什么	谁是权威
<code>gateway.query</code>	11	0ms	查支付网关的交易终态（轮询）	支付网关（外部）
<code>git-mcp</code>	7	126ms	仓库读写（MCP）	仓库本身
<code>gateway.refund</code>	6	0ms	向支付网关发起退款	支付网关（外部）
<code>sandbox.pytest_run</code>	2	334ms	在隔离沙箱里真跑 pytest	沙箱执行结果（外部判据）

§7.2 哈希完整性：93 组摘要逐条比对

```
$ python3 scripts/verify.py
[PASS] hash-integrity 93/93
    每个 skill / tool 调用的 input_digest / output_hash 与 event_log 一致
    失败 = 证据被篡改，或事后手写
```

93 = 67 次 Skill 调用 + 26 次 Tool 调用。摘要在调用发生的那一刻由被调用方写入，事后改任何一份产物的内容，重算出来的哈希就对不上 —— 这是「证据必须真实」这条铁律唯一可机器强制的部分。

§7.3 span 树：418 个 span，按 kind 分

kind	个数	这一层记的是什么
<code>event</code>	314	一条落库事件
<code>artifact</code>	55	
<code>task</code>	38	一个任务从派发到终态，挂在 plan 下
<code>plan</code>	11	一个计划的完整生命周期，是 span 树的根

22

模型调用（全部 estimated）

7,893

token 合计（估算口径）

0

归属不上的调用

这三个数字不能读成金额

缺省路径用 `ScriptedModelClient`，token 数是 `len(user)//4`（见 `maos/model/client.py`），非任何计费口径。核验器第 8 项校验的正是「estimated 标记与 model 列相符」—— 把估算印成真实计费，是这套观测面能犯的最坏的错。失败的模型调用单列且一个 token 数都不给：不知道就是不知道。本次 `failed_model_calls = 0`。

§7.4 trace 摘要里那四个「必须是 0」的字段

字段	八条链路合计	不是 0 意味着什么
<code>unsourced_artifacts</code>	0	有产物入了库却说不出是谁放的 —— 无名证据
<code>stray_event_count</code>	0	有事件挂不到任何 plan / task 上 —— 事件链有断点
<code>degraded_sandbox_reports</code>	0	沙箱降级跑出来的报告被当成了真跑的报告
<code>unrecorded_sandbox_reports</code>	0	沙箱跑过但报告没落库 —— 判据不可回查
<code>unattributed_model_calls</code>	0	有模型调用归不到任何计划上 —— 成本归因有黑洞
<code>tree_errors</code>	0	span 树有孤儿或成环 —— 这条链路的因果关系讲不通

这六个字段全是 0，而 `seeded_artifacts = 10` 不是 0 —— 旁路入库的产物确实存在，但每一份都被 `ArtifactSeeded` 点了名。「有旁路」和「有无名产物」是两件事，这份摘要要把它们分开记。

「所有 Agent 都回复完成」不等于业务成功。场景 6 与场景 7 是同一条链路、同一批 Agent、同一套闸——差别只在网关回了什么。

对照项	场景 6 · 顺利路径	场景 7 · 失败路径
Agent 自述	四个任务全部回复「完成」	四个任务同样全部回复「完成」
Plan 终态	DONE	FAILED (主管驳回)
业务状态	settled	compensated (全程没有经过 settled)
网关回执	code=10000 → observed_state=settled	40005 换渠道 → ACQ.SYSTEM_ERROR → outcome=unknown (问了 3 次仍非终态)
settled 观察	1 条, 带 actor_invocation_id	0 条 —— 没问出终态就一条都不该有
business_outcome	succeeded · basis=external_evidence	failed · basis=plan_failed · external_evidence 0 条
补偿记录	无	2 行: refund_request_revoked · manual_ticket
重规划	0 次	1 次 (40005 触发换渠道; ACQ.SYSTEM_ERROR 一票否决, 没有自旋)

§8.2 第七道闸的四象限：retriable 与 outcome 是正交的两维

retriable	outcome	disposition	严重度	为什么这么判
True	failed	replan_channel	blocker	网关在入口就拒了, 这一笔业务确定没执行 —— 可以换渠道重发
True	unknown	query_first	info	能再发一次, 但那一笔的下落网关自己说不清; 直接重发可能造成第二笔
False	failed	human_terminal	blocker	终态失败, 原样重发没有意义, 需转人工或改单
False	unknown	query_or_human	blocker	最危险的一档: 既不能原样重发、下落也不明, 必须先查或转人工

severity 与 disposition 必须留在同一张表里。拆开按 outcome 单独算一遍, 两套判据看的维度就不同, 必然在某一格分叉 —— 而分叉点恰好落在官方称「最危险」的 query_or_human, 症状是一笔下落不明的退款被静默放行: 不是红灯, 是没有灯。

unknown 不是 failed

域内补偿写的是「最后观察到的下落 = unobserved」, 不是 failed —— 网关自己都没给出结论, MAOS 凭什么替它下。

谁有权写 settled

核验器第 3 项钉死: 每个 settled 都必须有对应的 payment_observation, 且那条观察的 actor_invocation_id 必须属于 payment.observe。别的调用面写 settled, 这一项直接 FAIL。

§8.3 那两行补偿记录的原文 (库内 compensation_record)

kind	last_observed_state	detail 里写了什么
refund_request_revoked	unobserved	MAOS 侧不再推进本请求; 外部资金下落以 last_observed_state 为准, 需人工到支付渠道对账后确认 幂等键 rfd-tnt-mfg-001-case-s7-0001
manual_ticket	unobserved	渠道异常, 转人工 接单人已指派

铁律 8

MAOS 不持有权威事实, 只持有观察与推断。订单、支付、库存的权威状态永远归属外部系统。任何把外部状态直接写死为终态的代码都是 bug。这条不是文档里的一句话 —— 它是核验器第 3 项、第 6 项两道机器守卫。上面那行 meaning 是这条铁律写进数据里的样子: 作废的是本地意图, 不是「确认未付」。

R5 · RAG 有无对照。两段的计划脚本逐字节相同，唯一的变量是 `MAOS_KB_ENABLED=0 / 1` —— 所以差异只可能来自检索。

without_kb · MAOS_KB_ENABLED=0
4 个任务, KbRetrieved 0 条
第六道闸: blocker (漏排财务核算, 付款前拿不到金额凭据)
Plan **FAILED** · 业务状态 submitted

with_kb · MAOS_KB_ENABLED=1
5 个任务, KbRetrieved 1 条
第六道闸: pass
Plan **DONE** · 业务状态 settled

§9.2 两段的任务清单，差的就是那一个

#	without_kb (4 个任务)	with_kb (5 个任务)
1	受理多源退款诉求并聚合证据	受理多源退款诉求并聚合证据
2	按下单锁定的政策版本裁定退款资格	按下单锁定的政策版本裁定退款资格
3	发起退款并观察网关终态	核算退款金额并写财务分录
4	通知客户退款结果	发起退款并观察网关终态
5	—	通知客户退款结果

`delta_tasks = ['refund_finance.refund_finance']` —— 开了检索之后，计划里多出财务核算这一个任务。少了它，第六道闸在 plan 级就判 blocker: 闸连可判的对象都没有，因为漏排财务复核意味着没有任何任务带着申报金额。

§9.3 触发那次重排的文档（真实打分）

doc_id	标题	kind	总分	rule_no	fts	vector
kb-r5-history-0001	轴承锈蚀全额退款：财务核算不可省	history_case	0.719	1.00	1.00	0.845
kb-policy-tnt-mfg-a-AS-002-v1	质保期内质量问题全额退	policy	0.411	1.00	0.25	0.053
kb-policy-tnt-mfg-a-AS-004-v1	经销渠道差异（增渠道商核销，区域经理审批）	policy	0.150	0.00	0.75	0.000

检索漏斗（实测）
库存 17 条 → 同租户 9 条 → 七维预过滤后 5 条。按 kind 分: history_case 1 条 / policy 16 条。语料根 scenarios/refund。

排第一的那条知识是自己长出来的
kb-r5-history-0001 由本库一条真实收口成功的 case 晋升而来: case_id=case-r5-hist · biz_status=settled · 客户 ack 1 条。核验器第 7 项校验的正是这条追溯链。

§9.4 关掉检索之后，系统是怎么收场的

观测点	without_kb	with_kb
第六道闸结论	blocker	pass
Gate 总结论	blocker	pass
KbRetrieved 事件	0	1
财务分录	0	1
失败任务	—	—
业务状态	submitted	settled

第三出口：机器返工修不好的，转人工
受理多源退款诉求并聚合证据 → **BLOCKED** (await=human_decision)，主管裁决驳回，计划收敛到 FAILED —— 付款任务一次都没派发。闸既没把一笔缺金额凭据的退款放出去，也没在原地自旋返工。

上面八条链路证明的是「机器这一侧站得住」。这一页证明人在链路里的那个停点是真的 —— 五岗圆桌在真 Matrix 房间里演四种结局，不是四条 if 分支。

§10.1 四种结局各演一单（5 单实跑）

行	订单	裁定	预演金额	该谁拍板	阻断项	收口话术
2	ORD-2026-0004	批准 approve	9600.00	supervisor	0	建议批复 · 核准预演 9600.00 · 请 supervisor 拍板
3	ORD-2026-0005	补件后再议 need_more	1980.00	supervisor	4	需补件后再议 · 证据：缺少 image 类证据
4	ORD-2026-0006	升级审批 escalate	88000.00	finance_manager	5	建议升级审批 · 风险 high · 请 finance_manager 复核
5	ORD-2026-0006	升级审批 escalate	88000.00	finance_manager	4	建议升级审批 · 风险 high · 请 finance_manager 复核
6	ORD-2026-0005	驳回 reject	—	supervisor	2	不建议批复 · AS-001@v1 窗口 30 天，第 58 天申请，58 > 30

四种结局全覆盖：批准 / 补件后再议 / 升级审批 / 驳回。同一个订单 ORD-2026-0006 连着两行都判 escalate —— 重复退款的信号是从底账里查出来的，不是写死的。

§10.2 阻断项来自五个岗位各自的核验，不是一句话结论

ORD-2026-0006 · 建议升级审批 · 风险 high · 请 finance_manager 复核

- 风险：订单 ORD-2026-0006 在退款底账里已有处理中或已退成的记录，属重复退款申请
- 风险：订单 ORD-2026-0006 已有一笔退款到账 (settled)，本次是对同一单的二次退款
- 风险：同一客户近 30 天内已有 3 笔退款记录，频率偏高
- 风险：申报金额 92000.00 高于订单实付 88000.00 (比值 1.05)，核算会按实付封顶
- 风险：同一客户名下有 3 笔订单快照，存在批量退款的可能

五岗：申请受理岗 · 规则审核岗 · 证据核验岗 · 风险反欺诈岗 · 财务执行岗。每岗的结论都带[事实卡]，说得依据的字段与出处。

§10.3 真房间侧的实跑证据

用例	入口	终态	房间消息
R1 顺利路径	hiclaw.room_demo --case approve	task=DONE plan=DONE, exit=0	23 条
R2 失败路径	hiclaw.room_demo --case reject	task=FAILED plan=FAILED, exit=0	18 条
越权用例	含在 R1 那一轮里	两次「无审批权限」，闲聊零回复	见截图 05

房间证据的载体是截图

evidence/room/ 下 5 张实拍：审批卡片、状态迁移、/approve 的效果、/reject 后的补偿、名单外账号被拒。采集人手写，不是脚本产物 —— 这一条在该目录 README 里明说，并已记进 docs/DECISIONS.md。同目录的 transcript.md 是例外：它是从房间消息历史整份拉下来的脚本产物。

脱敏口径

homeserver、room_id 一律 <redacted> 不入库。任何需要真实密钥的配置只读环境变量 —— 密钥不许出现在 evidence/ 的任何输出里，凡是可能回显 env 或 URL 的命令，输出前必须过脱敏。

这一页是给核验者的。从零开始，两条命令，约 7 秒，你会拿到与本材料里每一个数字一致的结果。

§11.1 复现步骤

```
# ① 取到本材料的代码基线
$ git clone <repo> && cd MAOS && git checkout bcd5d46

# ② 产证据（跑 7 个场景 + RAG 对照，写 evidence/scenario-1..7 与 -R5）
$ python3 scripts/make_evidence.py

# ③ 逐项重放核验
$ python3 scripts/verify.py
RESULT: 8/8 PASS

# ④ 存量测试（与 ③ 是两条独立的绳子）
$ python3 -m pytest maos/tests -q
1935 passed, 41 skipped in 52.17s

# ⑤ 看任意一条链路的完整过程（Scripted 模式，无需任何 key）
$ python3 run.py --scenario 7
```

③ 不能单独跑。*.db 不入 git，核验器要的是库不是快照 —— 跳过 ② 直接跑 ③ 会报「缺数据库」并退出 2。这是设计行为。

§11.2 每条链路留下的文件

文件	是什么	对应核验项
result.json	Plan / 任务 / 终态 / metrics / business_outcome 的导出	第 6 项 business-outcome
trace.json	span 树 + 成本视图 + 摘要（schema maos.trace/v1）	第 4 项 trace-tree · 第 8 项 cost-attribution
run.log	跑那一次的终端原始输出，含完整状态迁移轨迹	人读；与库互为佐证
business-objects.json	每条 business_ref 指向的业务对象快照	第 2 项 business-ref
kb-hits.json	每次 KbRetrieved 的候选文档与多通道打分	第 5 项 kb-hit
maos.db	那一次运行的完整 SQLite 库（不入 git，由 ② 生成）	第 1 / 3 / 7 项的原始依据
dag-diff.json	仅 R5：两段 DAG 的差异与检索漏斗	第 5 / 7 项

§11.3 本材料的可核验性声明

数字的出处

全文 没有手打的计数。每一个 span 数、事件数、调用次数、状态迁移条数，都由生成脚本从 evidence/*/maos.db 与同目录 JSON 直接读出。证据重跑后数字会跟着变 —— 这正是想要的行为。

合成数据的边界

政策与历史案例是按行业惯例构造的合成数据；支付网关的错误码与时序取自公开规范。合成的是业务数据，不是运行记录 —— 运行记录是真跑出来的。

代码基线 bcd5d467f63b310dafeff4de38a351e9ca0a7d70
证据基线 cfe438475f751bb1a7bfe034daf7b2381314888d
核验 RESULT: 8/8 PASS | exit=0
测试 1935 passed, 41 skipped in 52.17s
链路合计 8 条链路 · 11 Plan · 38 任务 · 163 迁移 · 418 span · 314 事件
调用合计 67 次 Skill (13 个) · 26 次 ToolPort (4 个) · 55 份产物 · 35 条业务对象引用
生成于 2026-09-03T16:11:54+08:00